

Assignment 1

Search Constraint Satisfaction and Games
Dr. Brent Davis

You will compare search procedures, formulate and diagnose constraint models, and reason about an opponent before implementing two short game-tree functions. This is an individual assignment for all three course sections. Total: 100 marks; worth 5% of the final course grade.

Submission	Requirement
Stated deadline	Friday October 9, 2026 at 11:55 p.m. Eastern Time
Firm cutoff	Sunday October 11, 2026 at 11:55 p.m. Eastern Time
Late period	One automatic 48-hour no-penalty period, as specified in the course outline.
Submit on Brightspace	One ZIP containing answers.pdf, student_search.py, and results.json.
Report contents	Label A1–D3. Include readable traces, diagrams, calculations, and the Part D visualization.

Workload and preparation

Plan for approximately 5–7 hours after the relevant teaching and reading. The estimates below total 5.5–6.75 hours; experience and debugging time will vary. The Python component uses the standard library only.

Part	Marks	Estimated time
A Graph search and heuristics	25	75–90 min
B Constraint modelling and inference	25	75–90 min
C Games and uncertainty	30	85–100 min
D Python and interpretation	20	75–95 min
Setup and final review	—	20–30 min

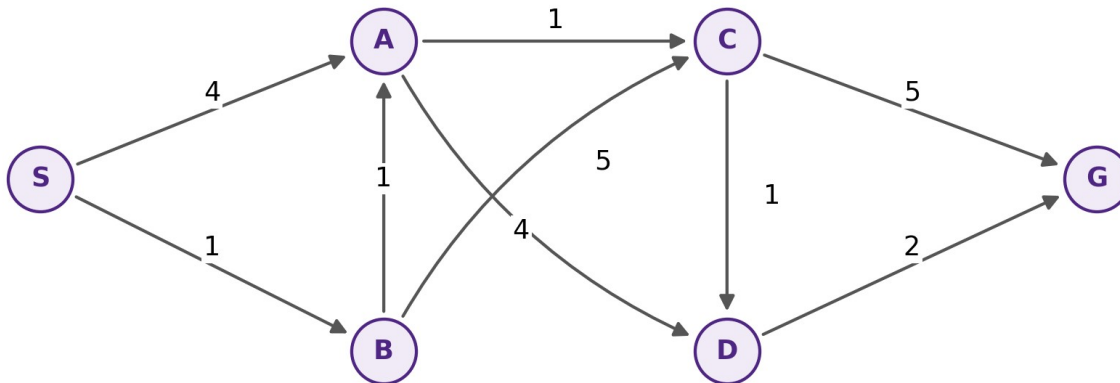
Authorship and evidence

Complete the work individually. You may use course materials and Python documentation, citing any outside sources. Generative AI is prohibited unless Dr. Davis announces a specific exception in class and confirms it in writing on Brightspace. No exception is granted by this handout.

Show your reasoning. A final number or screenshot alone does not satisfy a trace or explanation question. Work through Part C on paper before using your code to check it. Legible hand-drawn diagrams are acceptable. If coding is incomplete, submit partial work and identify which results you could not produce.

Part A Search problem data

A robot must travel from S to G. The graph is directed; a crossing between lines is not a state. Edge weights represent travel cost. The figure and adjacency table define the same graph. Use the table if a label is difficult to read.



State	Outgoing neighbors and costs	h
S	A:4, B:1	6
A	C:1, D:4	0
B	A:1, C:5	5
C	D:1, G:5	0
D	G:2	2
G	None	0

Required search conventions

Start with S at path cost $g=0$. Test for the goal when an entry is removed from the frontier, after discarding stale entries. Report valid removal order including G; do not include stale or duplicate removals that are skipped.

DFS and BFS: mark a state reached when it is first removed. Do not insert successors already reached. Pending duplicate entries may remain; skip them when removed if the state is already reached. Each entry retains its own path. BFS appends neighbors alphabetically. DFS pushes neighbors in reverse alphabetical order so the alphabetically first successor is considered first.

UCS and A*: retain the best known g for each state and update only for a strictly smaller g . Discard superseded entries. Break equal priority alphabetically by state name. UCS uses priority g ; A* uses $f=g+h$. For A3, a strictly better g reopens a previously removed state. A4 explicitly changes that rule.

Part A Questions 25 marks

A1 [9 marks] Run DFS, BFS, and UCS. For each, provide the valid removal order, returned S-to-G path, and total cost. For UCS, also show the active frontier after each non-goal removal, in priority order, as state:g. Show the cost sum for each returned path.

A2 [6 marks] Compute the true cheapest remaining cost h^* for every state. Is the supplied h admissible? Is it consistent? Justify your answers and identify every directed edge that violates $h(u) \leq \text{cost}(u,v) + h(v)$.

A3 [6 marks] Run A* with reopening. Give one trace row per valid removal: state, g, h, f, and active frontier after expansion. Write frontier entries as state:g/f. Mark reopened states and give the final path and cost. Stop at the first valid removal of G.

A4 [4 marks] Repeat the reasoning with a permanent closed set: discard an improved path if its successor is already closed, without changing that state's g or parent. State the returned path and cost. In at most 150 words, explain why admissibility alone does not rescue this implementation and why reopening changes the outcome.

Trace notation

A frontier entry X:4/7 means the current path to X costs 4 and its priority f is 7. List only current entries, not superseded heap records. At the goal row, write STOP; do not expand G. A repeated state name in the A* trace can be correct when an improved route causes reopening.

What to submit for Part A

A results table for DFS/BFS/UCS; a UCS frontier table; a table of h and h^* plus the consistency checks; the A* trace; and the A4 explanation. Search-tree drawings are optional here because the trace tables make the duplicate-state rules explicit.

Part B Constraint satisfaction 25 marks

Four robotics tests A, B, C, and D each require one full time slot on one exclusive test rig. The slots are numbered 1–4. A must finish before B starts, and C must finish before D starts. Every test must run exactly once, and no two tests can occupy the same slot.

Test variable	Allowed slots
A	{1,2}
B	{2,3}
C	{1,2,3}
D	{3,4}

B1 [6 marks] Formulate the CSP with variables, domains, and all constraints. Draw the binary constraint graph. Express the exclusive-rig condition as pairwise inequalities, labelling precedence on the appropriate edges as well.

B2 [7 marks] Use backtracking with forward checking to find the first solution. Choose an unassigned variable by minimum remaining values (MRV), then largest degree among unassigned neighbors, then alphabetical name. Order values by the number of values they remove from all unassigned-neighbor domains (least constraining value, LCV); break ties by smaller slot. Show the initial choice, the domains after every assignment, and the LCV removal counts for B after A=1.

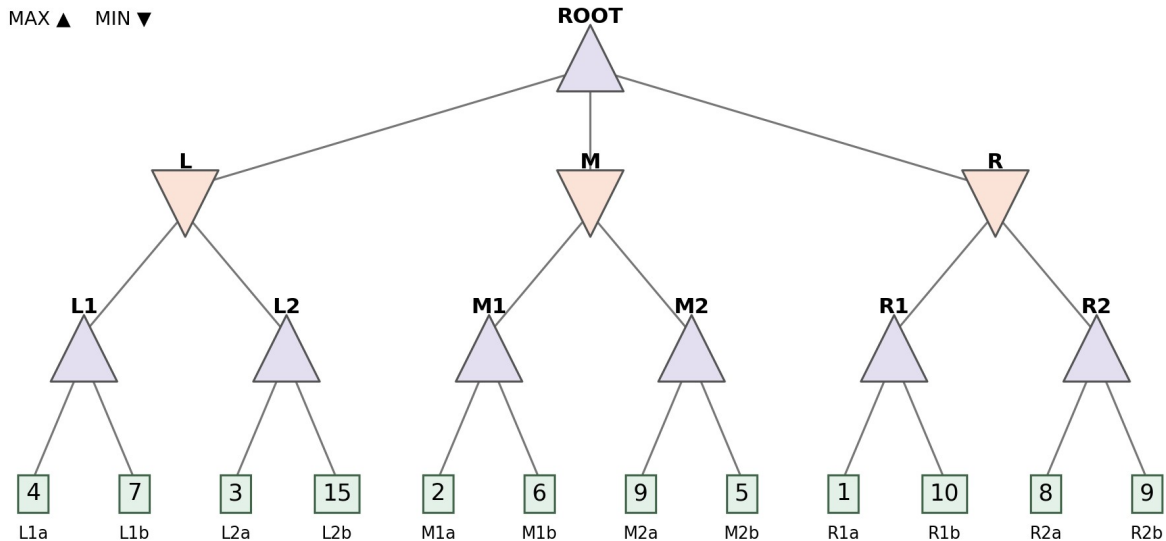
For B2, forward checking applies the newly assigned variable's constraints to its unassigned neighbors only. Do not run initial AC-3, propagate between two unassigned variables, or automatically assign singleton domains. Count each removed neighbor-domain value once, even if two constraints exclude it. Keep assigned domains as singletons in your trace.

B3 [4 marks] List every complete feasible assignment in (A,B,C,D) order. Explain briefly why your list is exhaustive; do not claim that finding the first solution proves uniqueness.

B4 [8 marks] Consider a separate modified instance: three tests X,Y,Z, domains {1,2} for each, and $X \neq Y$, $X \neq Z$, $Y \neq Z$. Run binary arc consistency before any assignment and state the resulting domains. Decide whether a solution exists and justify the decision. Then show forward checking after $X=1$ and the next assignment selected by alphabetical tie-breaking. Identify the exact domain wipeout. Explain what this shows about arc consistency and global feasibility.

Part C Game data and pruning 30 marks

A planning agent chooses L, M, or R. In the base model its opponent minimizes the agent's utility; the agent moves again at the third level. All leaves are terminal utilities from MAX's perspective. ROOT and L1–R2 are MAX; L,M,R are MIN.



MAX node	Left leaf a	Right leaf b
L1	4	7
L2	3	15
M1	2	6
M2	9	5
R1	1	10
R2	8	9

C1 [6 marks] Calculate every internal-node minimax value, showing the min/max operation at each node. Give the root value and its optimal move.

C2 [12 marks] Run alpha–beta left-to-right, then right-to-left at every internal node. Begin with $(-\infty, +\infty)$. Prune when $\alpha \geq \beta$. Use fail-soft returns: return the best value observed, even if it lies outside the incoming window. After each child returns, record node, child, incoming window, returned value, current best, updated window, and whether a cutoff occurs. List evaluated leaves in order and pruned leaves; compare leaf counts. Explain in at most 150 words why the root value stays the same and why a cutoff return may only be a bound.

Part C Opponent models and utility

C3 [7 marks] Replace L,M,R by CHANCE nodes. Keep ROOT and L1–R2 as MAX. The probabilities below give the chance of the corresponding first or second child. Compute each chance-node value and the root's expected utility and chosen move. In at most 150 words, explain what assumption changed, how the move differs from minimax, and one risk of using this probability model against an adversarial opponent.

CHANCE node	First child probability	Second child probability
L	$P(L1)=0.40$	$P(L2)=0.60$
M	$P(M1)=0.25$	$P(M2)=0.75$
R	$P(R1)=0.80$	$P(R2)=0.20$

C4 [5 marks] A separate decision offers Safe with utility 6 for certain, or Risky with utility 0 or 10, each with probability 0.5. Choose by expected utility. Repeat after transforming terminal utility by $f(u)=u^2$. Explain why this increasing transform preserves deterministic minimax choices on this nonnegative domain yet can change expectimax choices. State the class of utility transforms guaranteed to preserve expected-utility preferences.

Reference definitions

At MAX, choose the largest child value. At MIN, choose the smallest. At CHANCE, use the supplied probability-weighted sum of child values. A distribution at a chance node describes the model's uncertainty; it does not establish that the opponent actually behaves that way.

Alpha is the inherited lower bound from MAX alternatives; beta is the inherited upper bound from MIN alternatives. These bounds are not generally the exact value of the node being visited. A cutoff can occur after the last child without saving any leaf evaluations. Count only leaves that the algorithm does not evaluate as pruned leaves.

Part D Python implementation 20 marks

The supplied Python files define the trees, validate small examples, run experiments, and create an SVG comparison. Implement only `alpha_beta` and `expectimax` in `student_search.py`. Python 3.10+ is sufficient; no external package or live service is needed. The two TODO regions should usually require around 25–40 lines in total, depending on style.

D1 [10 marks] Implement recursive alpha–beta with MAX/MIN nodes, inherited windows, fail-soft returns, and $\alpha \geq \beta$ cutoffs. Honor `reverse=True` at every internal node and append only evaluated terminal names to the caller’s visited list. Support negative utilities, single-child nodes, and supplied narrow windows. Do not hard-code tree names or answers.

D2 [6 marks] Implement `expectimax` for MAX/CHANCE trees using the supplied probabilities in child order. Evaluate all children, including zero-probability branches, so the trace convention is deterministic. Return exact expected utility up to normal floating-point rounding. Do not apply alpha–beta cutoffs at chance nodes.

D3 [4 marks] Run the public tests and `run_experiments.py`. Submit `results.json` and include `comparison.svg` in your report. In at most 150 words, compare the alpha–beta leaf counts, explain why the `expectimax` root value is not a pruning-correctness comparison with minimax, and report any discrepancy from your written results. If unfinished, document the failure rather than inventing output.

Provided file	Purpose
<code>game_tree.py</code>	Immutable Node model and the two assessment trees
<code>student_search.py</code>	The only file with assessed TODO regions
<code>test_public.py</code>	Five small practice tests; not the complete marking suite
<code>run_experiments.py</code>	Result JSON and visual comparison generator
<code>README.md</code>	Commands and submission instructions

Commands

```
python -m unittest -v test_public
python run_experiments.py
```

Every input is a valid, finite, acyclic tree. Each nonterminal has at least one child. At CHANCE, probabilities are nonnegative, match the number of children, and sum to one. Your functions must preserve the provided data and signatures, work on arbitrary valid trees of these kinds, and avoid global mutable state. You may add your own small tests.

Your written reasoning is marked independently from your code. The same arithmetic error is not repeatedly penalized when subsequent work applies a correct method consistently. See the course outline for the assignment-completion policy.